Lucas Iacono, Shafaq Siddiqi, Margarita Dubina, Ema Srna, Toni Guli
Graz University of Technology
Computer Science and Biomedical Engineering
Institute of Human-Centred Computing

3 Data Management SS26: Exercise 03 – APIs and Tuning

Published: Mon. 11.05.2026
Deadline: Tue. 02.06.2026, 11.59pm

- This exercise aims to provide practical experience with physical design tuning (such as indexing), as well as database APIs and connectors.
- The expected files specified in each task should be submitted to TeachCenter individually - *not zipped*. Make sure to adhere to the requested formats of results.
- **IMPORTANT:** This exercise must be completed individually. Any form of plagiarism, copying, or unauthorized collaboration will be sanctioned according to academic regulations. Strict measures will be taken against any violations.

3.1 Database Connectors (10/25 points)

In most practical use cases, after having developed a query, it is transferred in the business logic. Hence, the need arises for a class of libraries called database connectors, which bridge the gap between our applications (programs) and the database server. Although DBMS agnostic libraries exist (e.g. JDBC) in our case we will use a PostgreSQL specific connector in python called `psycopg`.

(a) Environment Set Up.
  - Install Python and Pip [1]
  - Create a *virtual environment* (`.venv`) [2]
  - Use Pip to install the package `psycopg` [3]

  No Deliverable

(b) Create a script, which connects to your local PostgreSQL instance, executes all queries from Assignment 2 and checks their correctness by comparing them with the reference solutions provided. In the end the script should output a CSV File with the following header:

```
Query Name, Row Difference Count\n
```

[1] `https://wiki.python.org/moin/BeginnersGuide/Download`
[2] `https://docs.python.org/3/library/venv.html`
[3] `https://pypi.org/project/psycopg/`

In this example the difference count is the number of rows diverging from the reference and the output of your query. The result set should be compared row by row, meaning it is sorting sensitive. The column name row (CSV header) should also be compared and counted. A template for this script can be seen in Listing 1.
You will not be graded on the row difference, so if you had some discrepancies in the second assignment you will not lose points, except if the row difference count is not correct.
Deliverables T3.1: Queries.py, QueryDiffs.csv

```
import psycopg
import csv
import os

# DB Credentials
# TODO: Change to your own.
HOST = 'my_hostname'
PORT = 5432
DB_NAME = 'my_db'
USER = 'me'
PASSWD = 'secret_pw'

# TODO: Change to your own.
QUERIES = [
  'SELECT * FROM some_table;',
  """
  SELECT *
  FROM other_table;
  """,
  '...'
]
INPUT_FILES = ['Q1.csv', '...']
OUTPUT_FILE = 'output.csv'

def diff(reference, other):
  # TODO: diff the two results.

def main():
  # TODO: Connect to the database

  for query in QUERIES:
    # TODO: Execute each query, compute and store diff.

  # Write all results to the file with the name OUTPUT_FILE.



if __name__ == '__main__':
  main()
```

Listing 1: Connector Template Script

3.2 Performance Testing an Index (15/25 points)

In this Task you are to iteratively create indices for the query found on Listing 2, and plot their runtime this must be done in a **Python script**. Feel free to rename the tables and columns depending on your schema.

```sql
SELECT
    c.customer_id,
    c.country,
    COUNT(DISTINCT s.sale_id) AS total_sales,
    COUNT(si.item_id) AS total_items,
    SUM(si.quantity) AS total_quantity,
    SUM(si.item_total) AS total_revenue,
    AVG(s.total_amount) AS avg_sale_value,
    MAX(s.sale_date) AS last_sale_date,
    MIN(s.sale_date) AS first_sale_date,
    SUM(s.total_amount) AS total_paid,
    COUNT(DISTINCT p.category) AS categories_bought
FROM customers c
     JOIN sales s ON c.customer_id = s.customer_id
     JOIN sales_items si ON s.sale_id = si.sale_id
     JOIN products p ON si.product_id = p.product_id
WHERE
    s.sale_date BETWEEN TIMESTAMP '2025-03-01'
                    AND TIMESTAMP '2025-05-01 23:59:59'
  AND c.country IN ('Germany', 'France', 'Italy', 'Spain', 'Austria')
  AND p.catalog_price > 50
  AND s.discounted = TRUE
  AND si.discounted = TRUE
GROUP BY c.customer_id, c.country
HAVING SUM(si.item_total) > 4
ORDER BY total_revenue DESC, total_sales DESC;
```

Listing 2: Query to be indexed

3.2.1 Details

- Your testing should include **at least 5 unique tests**. Meaning 5 different permutations of applied indices. Each test can have none, one or multiple indices of your choosing. Each index must be "used" in the query.
- The database must be reset after each test, in order to "guarantee" test independence. Meaning all previous non-default indices should be dropped, and the session should be reset (see `DISCARD`[4])
- After creating the indices for a test, the query should be run using `EXPLAIN` and `ANALYZE`, and return a `JSON` object.[5]
- The **Execution Time** of the query (see the returned json object) must be plotted for all tests.
- A line plot with the test "id" on the x-axis and execution time on the y-axis must be created.

[4] `https://www.postgresql.org/docs/current/sql-discard.html`
[5] Prepend `EXPLAIN (FORMAT JSON, ANALYZE)` to the query.

- The plot's x-axis does not need to be labeled, whereas the y-axis is labeled to represent the execution time in seconds.
- Use `matplotlib` for the plot.
- Tests "id" represent how rudimentary the index configuration is, meaning earlier iterations of your solutions should have a lower id. The expected result of the graph is a clear speeding up of the execution time.
- All the database operations (index creation/deletion, session resetting, query analyzing etc.) must be performed using the Python PostgreSQL API. Failure to do so will result in 0 points.
- A python script template was provided to you in Listing 3, note that you do not need to abide completely by it, but it would be a good starting point. **Important:** If you changed the query do not forget to include the updated version to the python script.
- The best test output (lowest exectution time) must be submitted in a JSON format (the output from `EXPLAIN`).

Deliverables T3.2: PerfTesting.py, PerfPlot.png, Plan.json

```python
import psycopg
import numpy as np
import matplotlib.pyplot as plt

# DB Credentials
# TODO: Change to your own.
HOST = 'my_hostname'
PORT = 5432
DB_NAME = 'my_db'
USER = 'me'
PASSWD = 'secret_pw'

# TODO: Change to your own.
# Test structure: (id, create step, drop step)
TESTS = [
  (1, ['CREATE INDEX ...'], ['DROP INDEX ...'])
]
QUERY = 'EXPLAIN (FORMAT JSON, ANALYZE) SELECT * FROM users;'

OUTPUT_FILE = 'plot.png'

def run_test(t):
  tid, create, drop = t

  for q in create:
    # TODO: Create each index

  # TODO: Run query, extract execution time and save it

  for q in drop:
    # TODO: Execute each reset query



def main():
  # TODO: Connect to the database

  for test in TESTS:
    run_test(test)

  # TODO: Plot the test results and save them to OUTPUT_FILE


if __name__ == '__main__':
  main()
```

Listing 3: Performance Testing Template Script